CS 0007: Introduction to Java Lecture 3

Nathan Ong
University of Pittsburgh
September 8, 2016

Primitive Types

NO, STRING IS NOT A PRIMITIVE

I Don't Want to Do Strings Anymore. Isn't There Anything Else?

- Strings are okay, but they don't do much for us.
- What about representing numbers and other things?
- We must use the building blocks of Java!

- **Primitive Values**:
  - `boolean`
  - `byte`
  - `char`
  - `double`
  - `float`
  - `int`
  - `long`
  - `short`

I Don't Want to Do Strings Anymore. Isn't There Anything Else?

- Most useful
  - `boolean`
  - `char`
  - `double`
  - `int`
- Second tier
  - `byte`
  - `long`

- **Primitive Values**:
  - `boolean`
  - `byte`
  - `char`
  - `double`
  - `float`
  - `int`
  - `long`
  - `short`

`boolean`

- Two values only:
  - `true`
  - `false`
- How do we declare one?

Type name = value;

```
boolean name = value;
```

```
boolean yes = true;
```

`char`

- Characters in single quotation marks
- How do we declare one?

Type name = value;

```
char mander = 'z';
```

`double`

- Decimal numbers (not whole numbers)
- When providing a value, it must have at least one decimal place (2 should be 2.0)
- How do we declare one?

```
double pi = 3.14;
```

`int`

- Integers (whole numbers)
- How do we declare one?

```
int zero = 0;
```

`byte`

- Special tiny whole number
- How do we declare one?

```
byte kiloB = (byte)127;
```

`long`

- Special larger whole number
- How do we declare one?

```
long timeAgo =
  123456789L;
```

`short`

- Special smaller whole number
- How do we declare one?

```
short y = (short)-4;
```

`float`

- Special smaller **`double`**
- How do we declare one?

```
float away = 3.5f;
```

Why is String not purple?

- Why is String not a primitive value?
- What is a String made up of?
- Characters **`char`**

WARNING

- **`char`** uses SINGLE QUOTATIONS
- **`String`** uses DOUBLE QUOTATIONS

- **`String`** can be empty! (**`""`** → legal)
- **`char`** CANNOT be empty! (**`''`** → illegal)

- Concatenation: + with only **`char`**`s` has a different behavior than with **`String`**`s`

Okay, Let's Do Some Stuff Now!

```
public class Name
{
  public static void main(String[] args)
  {
  String firstName = "Nathan";
  char midInitial = 'R';
  String lastName = "Ong";
  int age = 19;
  boolean hasFacebook = true;
  System.out.println(firstName + " " + midInitial +  " " + lastName +
    ", age: " + age +     ", has a Facebook: " + hasFacebook);
  }//end method main
}//End class Name
```

What is the output?

Wait Whaaaaa?

- How did he add all of those things together??? Is that even legal? Is he cheating? He must have hacked Java.
- Concatenation with any String forces other values to use their String representations!
- The value `19` is now `"19"`!
- The value `true` is now `"true"`!
- So again, what is the output?



Hmm, no one really says "has a Facebook: true"

Now what? How do we fix this?

Range of Values

- `boolean` – `false, true`
- `byte` – `-2^7, 2^7-1`
- `char` – **ASCII**
- `double` – -∞, +∞; with limited precision, and NaN (Not a Number)
- `float` – -∞, +∞; with limited precision, and NaN (Not a Number)
- `int` – `-2^31, 2^31-1`
- `long` – `-2^63, 2^63-1`
- `short` – `-2^15, 2^15-1`

This is great and all, but I really want to be able to store more useful info

- Feels limited if we have to stay with the Type name = value; structure
- And thus we can introduce a new structure

Type name = expression;

`boolean`

- Two values only:
  - `true`
  - `false`
- We can also assign the variable an expression to be evaluated.

`boolean` Operators

`&&` – Logical AND

`||` – Logical OR

`!` – Logical NOT

```
boolean needUmb = rain || cloud;
```

Parentheses can be used as a way to group

Truth Tables

A	B	A&&B
T	T	T
T	F	F
F	T	F
F	F	F

Truth Tables

A	B	A||B
T	T	T
T	F	T
F	T	T
F	F	F

Truth Tables

A	!A
T	F
F	T

Order of Operators

1. `!` – Logical NOT
2. `&&` – Logical AND
3. `||` – Logical OR

```
boolean needUmb = rain || !sun;
```

Conditional Logic

- “If … then …”
- Does not have an operator in Java

A	B	A□B
T	T	T
T	F	F
F	T	T
F	F	T

Conditional Logic

- We can use an equivalent statement to get the result we want.

A	B	A□B
T	T	T
T	F	F
F	T	T
F	F	T

Conditional Logic

- We can use an equivalent statement to get the result we want.

A	B	A□B
T	T	T
T	F	F
F	T	T
F	F	T

Conditional Logic

- We can use an equivalent statement to get the result we want.

A	B	!A||B
T	T	T
T	F	F
F	T	T
F	F	T